

Progree DevOps Internship

Task 2: Application Containerization & Asset Optimization

Intern: Haseeb Ullah

Environment: Ubuntu, Docker Engine, Docker Compose

Evidence captured: 30 September 2026 (UTC)

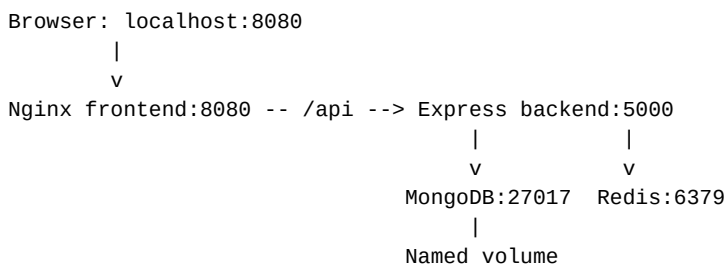
This project packages the existing Wanderlust travel blog into four cooperating containers. The DevOps contribution covers multi-stage images, configuration, networking, persistent storage, health checks, verification, and documentation.

Application attribution: Wanderlust by Krishna R Acharya and contributors, MIT license. The original application, license, and history are retained. This report does not claim authorship of the upstream web application.

Requirement coverage

Requirement	Delivered implementation
Multi-dependency runtime stack	React frontend served by Nginx, Express API, MongoDB, and Redis.
Multi-stage Dockerfiles	Frontend build/runtime stages; backend dependency/runtime stages.
Image footprint optimization	Alpine bases, production-only API dependencies, static frontend assets, narrow COPY rules, and .dockerignore files.
Secure environment configuration	Non-secret environment settings plus per-service, read-only secret files. Database application user is separate from root.
Functional port routing	127.0.0.1:8080 maps to the frontend; /api requests proxy to backend:5000. Database/cache ports stay internal.

Architecture



MongoDB and Redis use an internal Docker network. Only the backend joins both the web and data networks. The demo exposes only the frontend on localhost.

Build and runtime design

Frontend

Node installs locked dependencies and compiles TypeScript and Vite assets in a build stage. The final non-root Nginx image receives only the compiled dist directory and routing configuration. It supports direct client-side routes and compresses eligible responses. Browser requests use relative API URLs.

Backend

A dependency stage installs production dependencies with `npm ci --omit=dev`. The runtime stage copies only the dependency tree and required application files. Nodemon remains a development dependency. Node runs directly as a non-root user with a read-only root filesystem.

Runtime settings and credentials

Setting	Purpose
WEB_PORT=8080	Host port, configurable in the ignored root .env file.
MONGO_HOST / MONGO_DATABASE / MONGO_USER	Internal database connection metadata.
MONGO_PASSWORD_FILE	Read-only application database credential mount.
REDIS_URL / REDIS_PASSWORD_FILE	Internal cache URL and mounted password.
JWT_SECRET_FILE	Mounted signing secret for the email/password backend.
COOKIE_SECURE=false	Local HTTP demonstration; use HTTPS and secure cookies for a public deployment.

The setup script generates random credentials without displaying them. The .secrets directory is private on the host and excluded from Git. Compose mounts are local files, not an encrypted vault. The backend never receives the database root password. MongoDB initializes users only when its data volume is empty.

Reliability and application fixes

Docker health checks verify startup readiness. The API waits for its dependencies and closes connections on SIGTERM. MongoDB keeps data in named volumes; Redis remains disposable cache. An inherited cache-invalidation bug was reproduced and fixed so updated or deleted posts no longer remain in cached lists. Credential-bearing database URLs are excluded from logs.

Start and verify

```
python3 scripts/setup-local.py
docker compose up -d --build --wait --wait-timeout 180
python3 scripts/seed-demo.py
python3 scripts/verify-task2.py --recreate
python3 scripts/capture-evidence.py
```

Measured verification results

Comparison	Before (MiB)	Runtime (MiB)	Reduction
Backend baseline vs runtime	88.43	62.38	29.5%
Frontend build stage vs runtime	197.95	25.38	87.2%

Values are Docker image inspect .Size measurements from this engine, converted to MiB. They are not measurements of unique disk savings. The backend baseline includes development dependencies; the frontend comparison removes its build toolchain. Base images are pinned to tested digests. Raw image IDs and sizes are retained in evidence/image-sizes.json.

Functional checks

PASS: Frontend and direct SPA navigation respond

PASS: Backend, MongoDB, and Redis are ready through the Nginx proxy

PASS: Invalid post is rejected

PASS: Email/password API signup and signin use the runtime JWT secret

PASS: Create, read, and list a post backed by MongoDB

PASS: Update is visible in both detail and cached list routes

PASS: Post survives recreation of every project container using named volumes

PASS: Delete removes the post from MongoDB and cached lists

Configuration checks

PASS: All four services are healthy.

PASS: Only the frontend publishes a host port, bound to 127.0.0.1.

PASS: The database/cache network is internal.

PASS: Frontend, backend, and Redis use non-root users and read-only root filesystems.

PASS: Secret mounts are read-only; the backend cannot read the MongoDB root password.

PASS: MongoDB rejects unauthenticated data queries; Redis rejects unauthenticated PING.

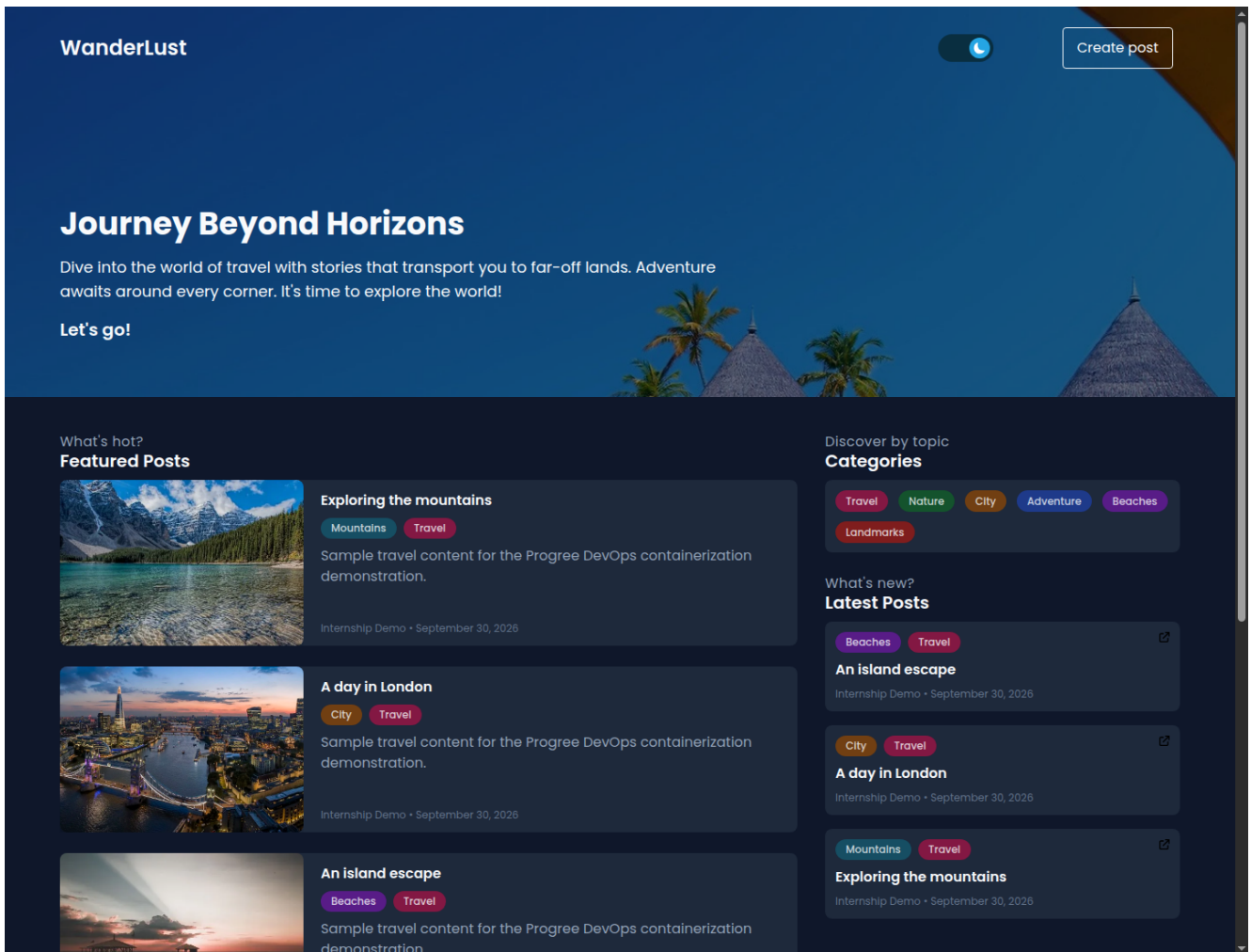
PASS: Generated secrets are absent from Git-eligible files and runtime image configuration/history.

Scope and limitations

This is a localhost containerization demo. The inherited post API has no authorization requirement. The upstream sign-in/sign-up screens are placeholders; backend email/password endpoints were verified separately. OAuth requires provider credentials and was not configured. Application dependency modernization and production security assessment are outside Task 2.

Application evidence

Homepage with database-backed demo posts



Captured from the running application at <http://localhost:8080> using headless Chrome. The rendered demo posts were loaded from MongoDB through the Nginx proxy and Express API.

Container status

Service	Health	Host access
backend	healthy	Internal only
frontend	healthy	127.0.0.1:8080
mongodb	healthy	Internal only
redis	healthy	Internal only

The recreation test removed and replaced all four containers while retaining named volumes, then retrieved the same updated post. Temporary test posts and accounts were cleaned up. Demo posts remain for review.

Create-post page and project handoff

Create Blog

Is this a featured blog? ☐

Blog title *

Travel Bucket List for this Year

Blog content *

Start writing here...

Author name *

Shree Sharma

Blog cover image (jpg/png/webp) *

https://...

Pick image

Categories (max 3 categories) *

Travel

Nature

City

Adventure

Beaches

Landmarks

Post blog

WanderLust

© 2026 All Rights Reserved

Direct navigation to /add-blog renders the React form, verifying Nginx's single-page-application fallback. API create/read/update/delete behavior is recorded in verification.txt.

Project and evidence locations

Local directory: /home/ubuntu/devops/progree-devops-internship

Portfolio repository name: progree-devops-internship

Guide: docs/task-2/README.md

Evidence: docs/task-2/evidence/

Stop while keeping data: docker compose down

Task 2 is complete based on the recorded checks. Tasks 3 and 4 are separate work. Combine this evidence with the other completed tasks into ONE final PDF for the organizer's Google Form by the stated deadline of 5 October 2026. This document has not been submitted.

References

Docker multi-stage builds: <https://docs.docker.com/build/building/multi-stage/>

Docker Compose secrets: <https://docs.docker.com/compose/how-tos/use-secrets/>

Original application: <https://github.com/krishnaacharyaa/wanderlust>